CUSTOMER SUPPORT TICKET ANALYZER

Data Analytics (DA) | Module-End Assignment – 4

Python-Based Customer Support Ticket Analysis

Project Report

1. Project Overview and Objective

1.1 Project Overview

This project presents a Python-based system for organizing and analyzing customer support tickets. Ticket details are stored in a dictionary of lists, and Python features are used to add records, validate user input, clean text, and produce summary statistics.

The workflow includes loading the initial dataset, adding new tickets, standardizing issue descriptions, counting selected keywords, reviewing ticket priorities, identifying the longest issue description, extracting unique words, and presenting the final results.

1.2 Project Objective

The purpose of this assignment is to apply practical Python programming and basic data-analysis methods to convert raw customer support information into organized data and useful analytical findings.

2. Data Sources

Item	Details
Dataset	Customer support ticket records
Domain	Customer Support / Service Operations
Initial Tickets	10
Additional Tickets	3
Final Tickets	13
Data Structure	Dictionary of lists
Development Environment	Jupyter Notebook
Programming Language	Python

3. Project Objectives

- Organize ticket information in a dictionary containing lists.
- Present the original ticket records in a clear, readable format.
- Allow users to enter new tickets with automatically assigned ticket numbers.
- Accept only High, Medium, or Low as valid priority values.
- Clean and normalize customer issue descriptions.
- Create a reusable function to count tickets containing specified words.

- Analyze the keywords poor, good, slow, and excellent.
- Summarize the number of tickets in each priority category.
- Identify the ticket with the greatest issue-description word count.
- Extract, count, and alphabetically sort unique words.
- Compile the results into a final analysis summary.

4. Attribute / Column Details

Attribute	Data Type	Description
Ticket_No	Integer	Unique identifier assigned to each ticket.
Customer_Name	String	Name of the customer associated with the ticket.
Issue_Description	String	Text explaining the customer's issue or feedback.
Priority	String	Ticket priority: High, Medium, or Low.

5. Tools and Technologies

- **Jupyter Notebook:** Used to write and execute the Python notebook.
- **Python:** Used for storing ticket data, input validation, text cleaning, analysis, and reporting.
- **GitHub:** Used to maintain the project repository and its documentation.

6. Data Setup and Processing

6.1 Initial Data Loading

- Loaded the 10 provided customer support tickets.
- Stored ticket attributes as lists within a dictionary.
- Printed the original ticket data for review.

6.2 Adding New Tickets

- Requested customer name, issue description, and priority for each new ticket.
- Generated ticket numbers sequentially, starting at Ticket 11.
- Added three new tickets, increasing the dataset size to 13 records.
- Validated priorities to ensure that only High, Medium, and Low were accepted.

6.3 Text Cleaning

- Converted issue descriptions to lowercase.
- Removed punctuation, including periods, commas, exclamation marks, question marks, and hyphens.
- Reduced repeated spaces to a single space.
- Removed leading and trailing whitespace.
- Standardized shorthand such as the standalone word 'ok' to 'okay'.

7. Python Analysis Structure

7.1 Keyword-Based Ticket Analysis

A reusable function named `count_tickets_with_word()` was created to count ticket descriptions containing a given keyword. The selected words were poor, good, slow, and excellent.

7.2 Priority Analysis

Loops and conditional statements were used to calculate the number of High-, Medium-, and Low-priority tickets.

7.3 Longest Issue Analysis

Each cleaned description was split into individual words. Word counts were compared to identify the ticket with the longest description.

7.4 Unique Word Analysis

A Python set was used to remove duplicate words from the cleaned descriptions. The distinct words were then sorted alphabetically and counted.

8. Core Analytical Tasks

Task	Description	Python Concepts
T1	Display the initial ticket data	Dictionaries, lists, printing
T2	Add tickets and validate priorities	Input, while loops, conditionals
T3	Clean issue descriptions	String methods, split, join
T4	Count selected keywords	Functions, loops, string comparison
T5	Analyze ticket priority distribution	Loops, conditional logic
T6	Find the longest issue description	Split, word count, comparison
T7	Extract unique words	Sets, loops, sorting
T8	Prepare the final report	Variables, functions, output formatting

9. Analysis Results and Key Insights

9.1 Overall Statistics

Metric	Reported Value
Total Tickets	13
High Priority	7
Medium Priority	3
Low Priority	3
Unique Words	41

9.2 Keyword Analysis

Keyword	Number of Tickets
Poor	3
Good	2
Slow	2
Excellent	1

9.3 Longest Issue Description

According to the supplied results, Ticket 2, submitted by Meera,, has the longest cleaned description: **" slow response very poor service"**. It contains 5 words.

9.4 Unique Word Analysis

The cleaned descriptions contain 41 distinct words. These were collected with a set, sorted alphabetically, and counted.

Note: The figures in this section reproduce the results provided for the assignment. They should be checked against the final notebook output, particularly if the three added tickets or the keyword-counting method differ.

10. Insights and Conclusions

10.1 Descriptive Analysis

- The final dataset contains 13 customer support tickets.
- Priority distribution is reported as 7 High, 3 Medium, and 3 Low tickets.
- A reusable function was used to analyze the four selected keywords.

10.2 Diagnostic Observations

- The keywords poor, good, and slow each appear in 3 ticket descriptions.
- The keyword excellent appears in 1 ticket descriptions.
- Ticket 2 has the longest reported description, with 5 words.

10.3 Practical Use

- Priority summaries can help support teams organize their workload.
- Keyword counts can highlight recurring terms that deserve further review.
- Description-length analysis demonstrates a basic method for examining text.
- Unique-word extraction provides a simple overview of the vocabulary used in tickets.

11. Code and Query Notes

- This assignment is implemented in Python rather than SQL; SQL queries are therefore not part of the project.
- The notebook performs the analysis using dictionary and list operations, loops, conditional statements, string methods,a reusable function, set operations, sorting, and formatted output.
- The executable Python code is maintained in the Google Colab notebook.

12. Project Deliverables

- Jupyter notebook containing the Python implementation.
- One-page project summary.
- GitHub README documentation.
- Final analysis report.

13. Skills Demonstrated

- Python data structures: dictionaries, lists, and sets.
- Iteration using for and while loops.
- Conditional logic using if, elif, and else.
- Function creation and code reusability.

- String cleaning and text processing.
- User input and validation.
- Keyword-based ticket analysis.
- Word counting and comparison.
- Set operations and sorting.
- Basic data analysis and report preparation.
- GitHub documentation and repository organization.

14. Conclusion

The Customer Support Ticket Analyzer demonstrates an end-to-end introductory data-analysis workflow using Python. It combines structured data storage, ticket creation, input validation, text normalization, keyword analysis, priority summaries, longest-description identification, unique-word extraction, and final reporting.

By converting raw ticket descriptions into a consistent format and summarizing important patterns, the project shows how fundamental Python techniques can support a clearer understanding of customer issues and service operations.

15. Author Details

Field	Details
Name	Angeline E
Course	Data Analytics (DA)
Assignment	Module-End Assignment – 4
Tool	Jupyter
Language	Python
Topic	Customer Support Ticket Analyzer